1. Frontend (Node.js with Express)

1. Create a frontend using Express and Node.js.
2. Include a form similar to the one from the Flask Assignment 2.
3. Configure the form to send a request to the Flask backend.

2. Backend (Flask)

4. Use the Flask backend to handle the form submission and process the data.

3. Folder Structure

5. Organize the project with separate folders for the frontend and backend.

4. Docker Configuration

6. Create a `Dockerfile` for both the frontend and backend.
7. Write a `.yaml` file (Docker Compose) to connect both services in the same network.
8. Upload both images to Docker Hub, and push your whole code to GitHub. Add node_modules and other non-required files (e.g., `.vscode`) to .gitignore.

Reference: https://github.com/Toto3107/flaskp

1. Project Overview

This project demonstrates a containerized full-stack web application. The frontend is built with Node.js and Express, while the backend uses Python and Flask. Docker Compose builds and runs both services on a shared bridge network.

2. Objectives

Create separate frontend and backend applications.

Build a Docker image for each service.

Connect the services using Docker Compose networking.

Test the form submission from the browser through the frontend to the Flask backend.

Publish both images to Docker Hub.

Prepare the project for submission through GitHub and a ZIP archive.

3. Technologies Used

Node.js and Express — serves the frontend and forwards form requests.

Python and Flask — validates the submitted name and email and returns JSON.

Docker — packages each service into an image.

Docker Compose — builds and runs both services together.

PowerShell — used to run Docker and Git commands.

4. Project Structure

```
Docker_Vikas/
├── frontend/
│   ├── public/
│   │   └── index.html
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── screenshots/
│   └── form_submission_success.png
├── compose.yaml
├── .gitignore
├── README.md
└── Docker_Assignment_Documentation.docx
```

5. Frontend and Backend

Frontend (Node.js + Express)

The frontend serves the HTML form on port 3000. When a user submits the form, the browser sends JSON to the Express endpoint /api/submit. Express forwards the request to the backend service at http://backend:5000/submit and returns the backend response.

Backend (Python + Flask)

The Flask application listens on port 5000. Its / route returns a health/status message. Its /submit route accepts POST requests, checks that name and email are present, and returns a success response or a validation error.

6. Dockerfiles

Backend Dockerfile

Uses python:3.12-slim as the base image, sets /app as the working directory, installs Flask from requirements.txt, copies app.py, exposes port 5000, and starts the application.

Frontend Dockerfile

Uses node:20-alpine as the base image, sets /app as the working directory, installs production dependencies from package.json, copies the Express server and public files, exposes port 3000, and starts the server with npm start.

7. Docker Compose and Networking

The root compose.yaml defines two services: backend and frontend. Each service is built from its own folder and attached to the app-network bridge network. The frontend depends on the backend service. Docker's internal DNS allows the frontend to reach the backend by the service name backend at port 5000.

Published host ports:

Frontend: http://localhost:3000

Backend: http://localhost:5000

8. Commands Used

Build and start both services: `docker compose up --build -d`

Check running containers: `docker ps`

View images: `docker images`

View service logs: `docker compose logs`

Stop and remove Compose containers/network: `docker compose down`

Initialize Git repository: `git init`

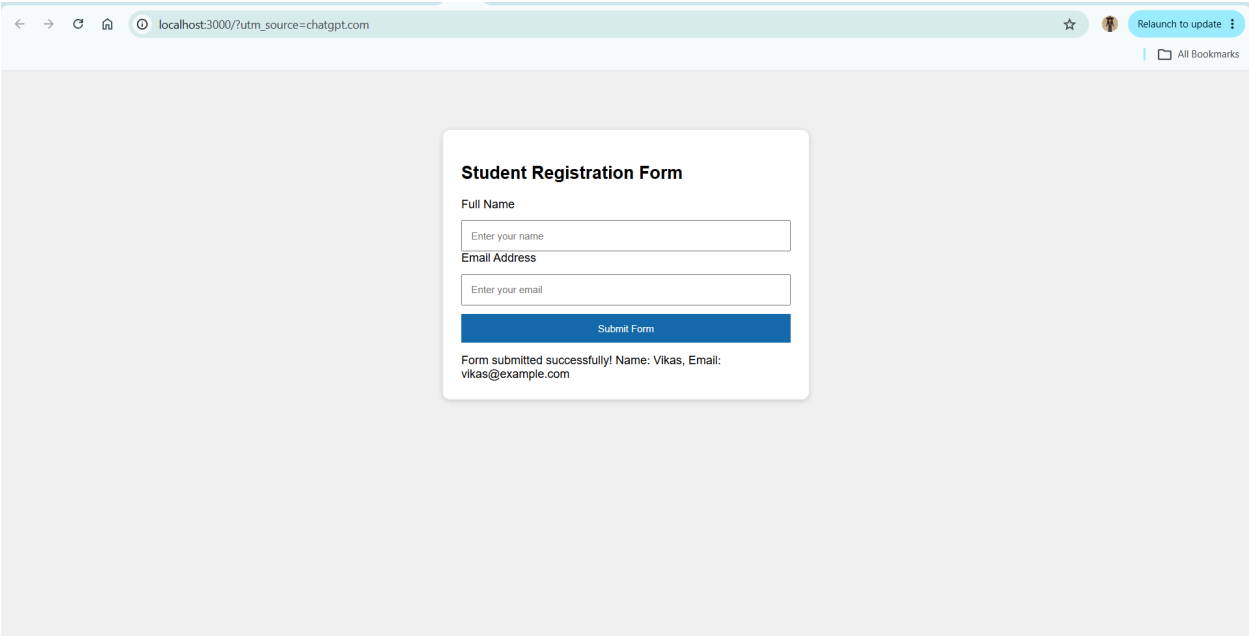
9. Testing and Result

The frontend was opened at http://localhost:3000 and the form was submitted with a name and email. The frontend successfully forwarded the request to Flask, and the page displayed a successful submission message. Backend connectivity was also checked from inside the frontend container; the backend returned HTTP status 200.

10. Screenshot Evidence

The successful form-submission screenshot is saved locally at:
Docker_Vikas\screenshots\form_submission_success.png

Note: The screenshot itself was not available in this chat workspace, so this document records its expected location. To embed it in the Word document, open this file, place the cursor below this paragraph, choose Insert > Pictures, and select screenshots\form_submission_success.png.



11. Docker Hub Images

Backend image: vvera4739/vikas-flask-backend:1.0

Frontend image: vvera4739/vikas-node-frontend:1.0

Both images were successfully pushed to Docker Hub.

12. GitHub Repository

GitHub repository URL: TO BE ADDED AFTER THE PROJECT IS PUSHED TO GITHUB.

13. Conclusion

The project demonstrates containerization of a Node.js frontend and Flask backend, communication between services through a Docker Compose network, successful form submission, and publishing Docker images to Docker Hub. The GitHub repository URL should be added after the source code is pushed.